SKYWAY AIRLINES

Airline Reservation & Management System

Comprehensive Project Guide, File-by-File Code Walkthrough,
Database Concurrency Architecture & Viva Voce Defense Preparation

Domain & Category:	Full-Stack Enterprise Desktop Application & Mobile QR Gateway
Core Language & GUI:	Java (JDK 25) • Java Abstract Window Toolkit (AWT) & 2D Graphics
Relational Database:	MySQL Server (ACID Transactions, Row Locks via JDBC Driver 8.4)
Integrated Libraries:	Apache PDFBox 3.0, Jakarta Mail 2.1, Google ZXing 3.5, React/Vite
Student Presenter:	Avadhoot Savle
Target Evaluation:	College Project Submission, Code Inspection & Viva Voce Examination

■ **Objective of this Document:** This guide breaks down every module, algorithm, and line of code in the SkyWay system. It explains *what* each component is doing, *why* it was engineered that way, and *how* to present it convincingly to professors and external examiners during lab evaluation tomorrow.

1. System Architecture & End-to-End Workflow

SkyWay is an enterprise-grade desktop airline reservation application that marries classic low-level Java GUI development with modern distributed transaction patterns. The application avoids bloated third-party GUI frameworks in favor of native **Java AWT (Abstract Window Toolkit)**, combining it with high-performance 2D canvas rendering, a built-in micro HTTP web server, automated vector PDF boarding pass generation, and encrypted Gmail SMTP integration.

Layer	Tech Stack	Responsibilities & Capabilities
Presentation (GUI)	Java AWT (Frame, Canvas, Graphics), Custom SeatMap	Desktop windows, coordinate-based aeroplane fuselage map, real-time seat availability color-coding, JVM-wide Ctrl+Key shortcuts.
Business Logic	AirlineMain, FlightFormFrame, PassengerDetailsFrame, TicketService	Flight scheduling, seat generation algorithms, fare calculation, 6-character PNR generation, PDF compilation.
Mobile Gateway	ZXing QR Library, com.sun.net.httpserver.HttpServer, React/Vite	Generates dynamic QR codes, detects laptop LAN IP, serves HTTP webhooks (port 8085) for mobile phone simulated checkout.
Data & Concurrency	MySQL 8/9, JDBC Connector 8.4, DBConnection	ACID transaction boundary, pessimistic row-level locking (`SELECT ... FOR UPDATE`), 6 normalized tables in 3NF.

End-to-End Operational Flow Diagram

CUSTOMER BOOKING & PAYMENT FLOW:
[Customer Login] → [AvailableFlightsFrame] (Select Flight) → [SeatSelectionFrame] (Click on 2D Aeroplane Canvas) → [PassengerDetailsFrame] (Enter Name, Age, Gender) → [PaymentFrame] (Displays ZXing QR Code) → **[Phone Scans QR → React Webpage]** → Click 'Payment Done' → **[PaymentServer (Port 8085)]** receives webhook → **[MySQL ACID Transaction]** (Locks Flight → Marks Seats 'BOOKED' → Saves Booking) → **[TicketService]** (Apache PDFBox generates PDF → Jakarta Mail sends email).

2. Database Design & Concurrency Control

The relational database schema is normalized to 3NF to avoid update anomalies and guarantee ACID compliance. It consists of 6 primary tables in the airline_db database:

Table	Keys & Constraints	Columns & Purpose
admins	PK: admin_id	`username` (UNIQUE), `password`. Stores administrator login credentials.
customers	PK: customer_id	`name`, `email` (UNIQUE), `phone`, `password`, `created_at`. Registered passenger profiles.
flights	PK: flight_id	`flight_no`, `airline`, `source`, `destination`, `departure_time`, `arrival_time`, `price`, `total_seats`, `available_seats`.
seats	PK: seat_id FK: flight_id	`seat_number` (eg. 1A, 2C), `status` ('AVAILABLE', 'BOOKED'). Composite unique key on `(flight_id, seat_number)`.

bookings	PK: booking_id FK: flight_id, customer_id	`pnr` (6-char UNIQUE), `passenger_name`, `passenger_age`, `passenger_gender`, `seat_number`, `booking_date`, `status` ('CONFIRMED', 'CANCELLED').
payments	PK: payment_id FK: flight_id, customer_id	`payment_ref` (UNIQUE), `amount`, `status` ('PENDING', 'SUCCESS', 'FAILED'), `seats_csv`, `passengers_json`, `created_at`.

How Double-Booking is Prevented (Pessimistic Row-Level Locking)

A critical challenge in airline booking systems is **race conditions** (two users clicking 'Pay' for the same seat simultaneously). SkyWay solves this through a 3-step defensive locking mechanism inside PaymentFrame.java:

Step 1: Row Locking (`SELECT ... FOR UPDATE`):
Inside an explicit transaction (con.setAutoCommit(false)), we execute:
PreparedStatement ps = con.prepareStatement("SELECT available_seats FROM flights WHERE flight_id = ? FOR UPDATE");
This forces MySQL InnoDB to place an exclusive write lock on that flight row. Any other concurrent booking thread trying to book on that flight is queued until our transaction commits or rolls back.

Step 2: Atomic State Transition Check:
We update each seat with a strict status check:
UPDATE seats SET status='BOOKED' WHERE flight_id=? AND seat_number=? AND status='AVAILABLE';
If ps.executeUpdate() == 0, it means another thread claimed the seat a split second prior. The application immediately invokes con.rollback(), aborts the reservation, and alerts the user without corrupted data.

Step 3: Atomic Final Commit:
Only when all seats are locked, the booking records inserted, and the flight seat inventory decremented does con.commit() execute.

3. Complete File-by-File Code Walkthrough

Here is the granular, file-by-file breakdown of every Java source file in the application. Review each component's purpose, design, key methods, and presentation tips.

■ src/DBConnection.java

Role: Database connectivity provider with auto-detection of active MySQL port.

Key Methods & Implementation Logic:

- **getConnection()**: Loads `com.mysql.cj.jdbc.Driver`. Attempts connection to port 3306 first; if unavailable, automatically tries port 3307 (standard XAMPP port). Caches the successful port in `workingPort`.
- **workingPort (static Integer)**: Stores the verified port in memory so subsequent queries throughout the session connect instantly without re-testing.

■ **Viva Voce Tip:** Explain that hardcoding a single port breaks when moving between laptops with Homebrew MySQL (3306) and XAMPP (3307). Your code solves this with dynamic port discovery.

■ src/AirlineMain.java

Role: Application entry point; provides user role selection, global keyboard shortcuts, and payment server initialization.

Key Methods & Implementation Logic:

- **main(String[] args)**: Attaches a JVM-wide event listener to `Toolkit.getDefaultToolkit().addAWTEventListener(...)`. Starts `PaymentServer.start()` in the background, then instantiates the main frame.
- **eventDispatched(AWTEvent ev)**: Filters for `KeyEvent.KEY_PRESSED` with `isControlDown()`. When Ctrl+letter is pressed, it crawls the active window hierarchy and programmatically triggers the matching button's `ActionEvent`.
- **findButton(Container box, int key)**: Recursively searches nested panels inside the active window for any button whose label begins with the pressed character.

■ **Viva Voce Tip:** Demonstrate pressing `Ctrl+A` or `Ctrl+C` to your professor without touching the mouse. It proves mastery of low-level Java AWT event pipelines.

■ src/LoginFrame.java & src/RegisterFrame.java

Role: Role-aware credential authentication and new customer account registration.

Key Methods & Implementation Logic:

- **LoginFrame(String role)**: Configures GUI labels for either 'Admin' or 'Customer'. Queries `admins` table or `customers` table accordingly.
- **actionPerformed(ActionEvent e)**: Reads username/email and password fields. Executes `SELECT * FROM ... WHERE ... = ? AND password = ?` via `PreparedStatement`. Upon customer authentication, constructs `CustomerDashboard(id, name)`.
- **RegisterFrame submit**: Validates that name is non-empty, email contains '@' and '.', and phone number consists of exactly 10 digits before executing `INSERT INTO customers`.

■ **Viva Voce Tip:** Emphasize that `PreparedStatement` with parameterized placeholders (`?`) is used instead of string concatenation to guarantee immunity against SQL Injection.

■ src/AdminDashboard.java & src/CustomerDashboard.java

Role: Role-segregated navigation control centers.

Key Methods & Implementation Logic:

- **AdminDashboard()**: Provides navigation to `ManageFlightsFrame` and `ViewBookingsFrame`. Includes a safe session logout action.
- **CustomerDashboard(int id, String name)**: Displays personalized welcome banner ('Welcome, [Name]'). Directs user to `AvailableFlightsFrame` (booking) or `MyBookingsFrame` (booking history).

■ **Viva Voce Tip:** Note how session variables (`customerId`, `customerName`) are maintained and forwarded through constructors to associate subsequent bookings with the logged-in user.

■ src/ManageFlightsFrame.java & src/FlightFormFrame.java

Role: Admin flight scheduling and automatic seat inventory generator.

Key Methods & Implementation Logic:

- **loadFlights(String filter)**: Retrieves upcoming flights from MySQL, formatting flight number, airline, route, departure/arrival timestamps, fare, and seat count into an AWT `List`.
- **deleteFlight()**: Checks for active confirmed bookings before allowing flight deletion to prevent orphaned foreign keys.
- **saveFlight() in FlightFormFrame**: Inserts or updates flight record. If creating a new flight, it runs a batch insertion algorithm generating 30 seat rows (`1A` through `5F`) set to 'AVAILABLE'.

■ **Viva Voce Tip:** Explain the seat generator: whenever an admin creates a flight, 30 distinct seat records are automatically generated in the `seats` table.

■ src/AvailableFlightsFrame.java

Role: Customer flight discovery window with live inventory filters.

Key Methods & Implementation Logic:

- **searchFlights()**: Executes `SELECT * FROM flights WHERE departure_time >= NOW()` with optional source and destination text filters.
- **proceedToSeatSelection()**: Validates that a flight is highlighted and has `available_seats > 0`, then launches `SeatSelectionFrame` with the selected `flightId`.

■ **Viva Voce Tip:** Mention that flights with departure times in the past are automatically filtered out using `departure_time >= NOW()`.

■ src/SeatMap.java & src/SeatSelectionFrame.java (Custom 2D Vector Aeroplane Engine)

Role: Interactive 2D graphical aeroplane fuselage canvas allowing direct seat selection.

Key Methods & Implementation Logic:

- **paint(Graphics g) in SeatMap**: Draws the aeroplane fuselage from scratch: curved aerodynamic nose (`drawArc`/`fillArc`), cockpit windows, aircraft wings, central aisle walkway, and seat bounding boxes.
- **Color-Coded Status Engine**: Green (`#2ECC71`) = Available; Red (`#E74C3C`) = Booked; Blue (`#3498DB`) = Selected by customer. Renders armrests, headrests, and alphanumeric seat codes (`1A`, `2B`).
- **mouseClicked(MouseEvent e)**: Calculates hit-test collision: translates raw `(e.getX(), e.getY())` coordinates into matrix row and column. Enforces maximum 6 seats limit. Repaints canvas.
- **Selection Listener Callback**: Notifies `SeatSelectionFrame` to dynamically re-calculate and display total ticket fare in real time as seats are toggled.

■ **Viva Voce Tip:** THIS IS YOUR STRONGEST SHOWCASE: Most student projects use basic checkboxes. Your project implements a custom 2D vector graphics engine on an AWT Canvas with coordinate collision detection.

■ src/PassengerDetailsFrame.java

Role: Dynamic passenger manifest form generator and payment reference initializer.

Key Methods & Implementation Logic:

- **buildPassengerForms()**: Generates input rows (Name, Age, Gender Choice dropdown) matching the exact number of seats chosen in the seat map.
- **proceedToPayment()**: Generates unique payment reference (e.g., `PAY9K2X4B`). Inserts a pending record into `payments` table storing serialized passenger data, and transitions to `PaymentFrame`.

■ **Viva Voce Tip:** Explain that storing passenger data in `payments` before booking ensures uncompleted checkouts don't pollute the primary `bookings` table.

■ src/PaymentFrame.java & src/PaymentServer.java (Mobile QR Payment Gateway)

Role: Embedded HTTP micro-server and dynamic QR code payment gateway bridging phone to desktop.

Key Methods & Implementation Logic:

- **generateQRCodeImage(String url)**: Uses Google ZXing (`core-3.5.3.jar`) `QRCodeWriter` to encode `http://:8085/?ref=PAY...` into a 220x220 pixel buffered image.
- **PaymentServer.start()**: Spawns an embedded HTTP server on port 8085 using Java's built-in `com.sun.net.httpserver.HttpServer`. Handles `/api/payment/details`, `/api/payment/success`, and `/api/payment/fail`.
- **getLaptopIp()**: Scans active network interfaces via `NetworkInterface.getNetworkInterfaces()` to detect the laptop's Wi-Fi LAN IP so phones on the same network can load the payment portal.
- **completeBookingTransaction()**: Executes the atomic ACID transaction: row lock -> updates seat status -> generates 6-char PNR -> inserts booking rows -> decrements flight seats -> commits -> calls TicketService.

■ **Viva Voce Tip:** Highlight cross-device integration: the Java desktop app hosts its own HTTP server so an external phone can scan the screen and complete the transaction in real time.

■ src/TicketService.java (Automated PDF & SMTP Dispatcher)

Role: Vector PDF e-ticket compiler and asynchronous Gmail SMTP email dispatcher.

Key Methods & Implementation Logic:

- **generateTicketPDF(...)**: Uses Apache PDFBox 3.0 (`PDDocument`, `PDPage`, `PDPageContentStream`). Programmatically draws professional airline boarding passes with headers, route badges, PNR, passenger roster, and a verification QR code. Saves to `tickets/ticket_.pdf`.
- **sendTicketEmail(...)**: Reads Gmail credentials from `config.properties`. Connects to `smtp.gmail.com:587` with TLS encryption via Jakarta Mail. Attaches the generated PDF to a MIME multipart email.
- **Async Execution**: Dispatches the email on a background worker thread (`new Thread(...).start()`) so the desktop GUI remains silky-smooth and never freezes.

■ **Viva Voce Tip:** Point out the asynchronous threading: network operations like SMTP are offloaded to background threads to ensure optimal UI responsiveness.

■ src/MyBookingsFrame.java & src/ViewBookingsFrame.java

Role: Customer self-service reservation management and admin booking overview.

Key Methods & Implementation Logic:

- **cancelBooking(String pnr)**: Transactional cancellation: updates booking status to 'CANCELLED', resets seat status to 'AVAILABLE', and increments `available_seats` inside an atomic transaction.
- **openPDFTicket(String pnr)**: Uses `java.awt.Desktop.getDesktop().open(file)` to launch the generated PDF in the operating system's default viewer (Preview on Mac / Acrobat on Windows).
- **resendTicketEmail()**: Re-invokes `TicketService` to re-transmit the e-ticket to the customer's registered email.

■ **Viva Voce Tip:** Explain that cancellation uses transactional rollback: if updating the seat status fails, the cancellation is aborted to ensure database consistency.

■ src/MessageBox.java

Role: Lightweight native modal dialog component.

Key Methods & Implementation Logic:

- **MessageBox(Frame parent, String msg)**: Modal dialog that blocks parent input until closed, displaying formatted alerts and error notices.
- **Confirmation Mode**: Provides Yes/No confirmation options returning a boolean result, used before destructive operations such as flight cancellation.

■ **Viva Voce Tip:** Implemented using pure Java AWT primitives without requiring heavy Swing dependencies.

4. Viva Voce Defense & Examiner Q&A Cheat Sheet

Review these 8 high-probability questions before your defense. Deliver the provided model answers to demonstrate mastery of software engineering concepts:

Q1: Why did you choose Java AWT instead of Swing or JavaFX?

Answer: AWT gives direct, low-level access to native GUI primitives and lightweight graphics rendering. By extending `java.awt.Canvas` and overriding `paint(Graphics g)`, we built a fully custom 2D vector aircraft seating map with coordinate hit-testing that demonstrates foundational computer graphics principles rather than relying on heavy pre-built widgets.

Q2: How does your system guarantee that two passengers cannot book the same seat simultaneously?

Answer: We implement Pessimistic Concurrency Control inside an ACID JDBC transaction. Before updating, we lock the flight row using `SELECT ... FOR UPDATE` in MySQL InnoDB. Then we execute an atomic update with a status predicate: `UPDATE seats SET status='BOOKED' WHERE flight_id=? AND seat_number=? AND status='AVAILABLE'`. If `executeUpdate()` returns 0, another user claimed it; our transaction rolls back immediately.

Q3: How does the QR mobile payment function without an external cloud gateway?

Answer: Our Java desktop app embeds a lightweight HTTP server (`com.sun.net.httpserver.HttpServer`) on port 8085. We use the ZXing library to encode the laptop's Wi-Fi IP and payment reference into a QR code. When scanned on a phone on the same Wi-Fi, the phone opens our React web portal and sends an HTTP webhook back to our embedded Java server.

Q4: What is a PNR and how is it generated in your project?

Answer: PNR stands for Passenger Name Record. It is a 6-character alphanumeric unique identifier (e.g., `RB4TX2`) used globally in aviation to index all travelers under a reservation. We generate random uppercase alphanumeric strings and verify their uniqueness in the `bookings` table before confirmation.

Q5: Why did you use `PreparedStatement` instead of `Statement`?

Answer: `PreparedStatement` pre-compiles SQL queries on the database engine and binds parameters separately using typed placeholders (`?`). This completely prevents SQL Injection attacks and improves execution performance through query plan reuse.

Q6: How are email delivery failures handled without losing booking data?

Answer: The MySQL transaction commits *before* email dispatch. The email routine in `TicketService.java` runs asynchronously in a background thread inside a `try-catch` block. If the SMTP server is unreachable, the booking remains safely saved in the database, the PDF is preserved on disk, and the user can click 'Resend Ticket' in their dashboard.

Q7: What happens under the hood when a booking is cancelled?

Answer: Cancellation executes inside an atomic JDBC transaction. The booking status is updated to `'CANCELLED'`, the seat in the `seats` table is set back to `'AVAILABLE'`, and the flight's `available_seats` count is incremented by 1. All changes commit together so seat inventory is never corrupted.

Q8: What software architectural pattern does your database layer follow?

Answer: It follows the Singleton/Factory Pattern in `DBConnection.java`. It maintains a static connection factory and caches the active port (3306 or 3307) so the entire application shares an optimized, resilient connection pipeline.